

Finance Collections Agent

AI-Powered Accounts Receivable Automation

FastAPI · LangGraph · LiteLLM · Celery + Redis · Next.js 15

Author	Lakshya Rathi
Repository	github.com/laksh-ya/finance-follow-up
Demo Video	Linked in README
Default LLM	groq/llama-3.3-70b-versatile

1. Overview

Finance teams manually chase overdue invoices — inconsistent tone, no audit trail, delayed escalations, hours on copy-paste emails. This system automates the entire workflow: ingests invoices, classifies by days overdue, generates a personalised email in the correct tone using a strict LLM pipeline, routes risky drafts to human reviewers, dispatches via SMTP or Resend, and writes an immutable audit log after every action.

The system is built around a deterministic LangGraph state machine — not a free-form agent loop. Every invoice takes the same 10-node path, validation rules are enforced in code, every dispatch attempt is persisted, and the pipeline never fails silently.

Key Capabilities

- ▶ Stage-aware email generation — 4 escalation stages with distinct tone, key message, and CTA
- ▶ Hallucination prevention — LLM output cross-checked against source invoice in code before dispatch
- ▶ Human-in-the-loop — confidence-gated review queue with approve, edit, reject, regenerate, flag-legal
- ▶ Provider-agnostic LLM — Groq, Gemini, OpenAI, Ollama, Together — switched by env vars via LiteLLM
- ▶ Three email modes — mock log, SMTP sandbox (Mailtrap), live Resend — switchable at runtime
- ▶ Bidirectional Google Sheets sync — pull invoices in, write audit trail and status back
- ▶ Full observability — Langfuse LLM traces, Celery Flower, Neon DB, Upstash Redis dashboard
- ▶ Dead-letter queue with manual retry for permanently failed Celery tasks
- ▶ Everything live-configurable from Settings UI without a restart

2. Architecture

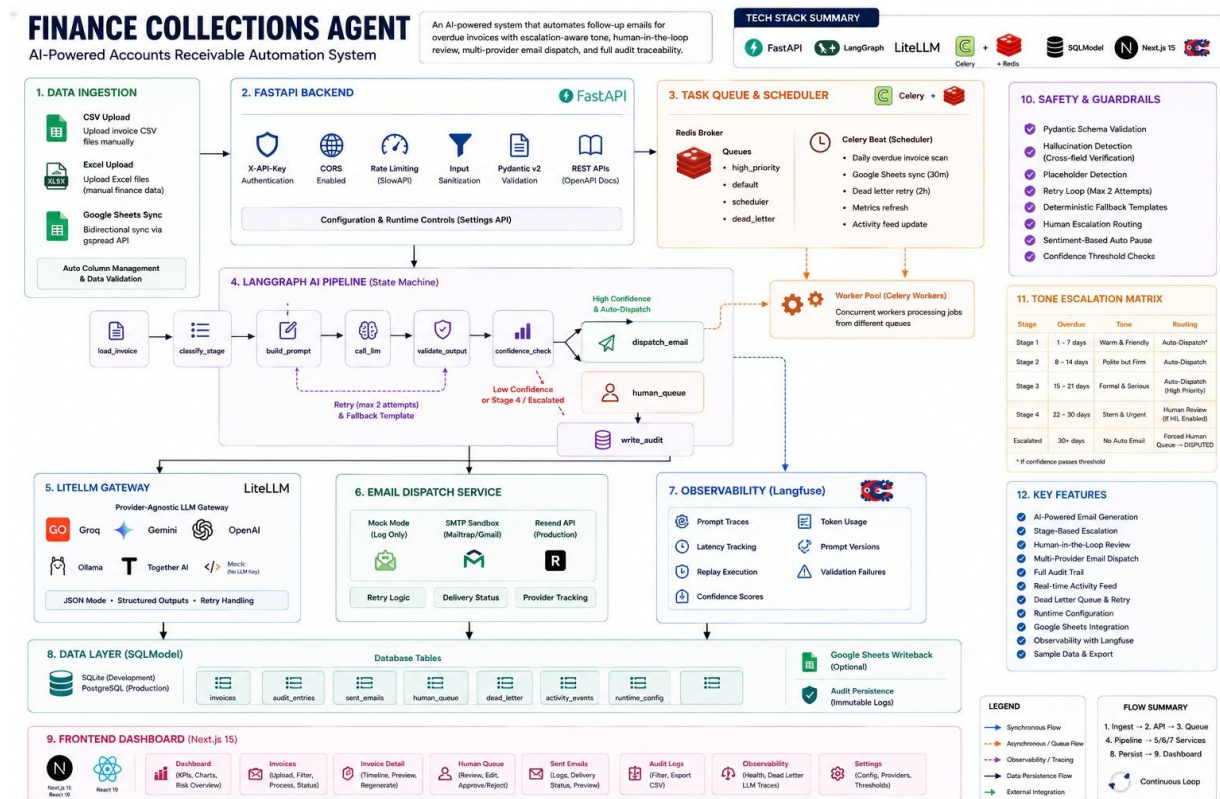


Figure 1 — Full system architecture across all 7 layers

System Layers

Layer	Technology	Role
Presentation	Next.js 15, React 19, Tailwind, shadcn/ui	8-page operator dashboard — live feed, human queue, observability, settings
API Gateway	FastAPI 0.115, Pydantic 2, SlowAPI	REST API with X-API-Key auth, CORS, rate limiting, input sanitisation
Orchestration	Celery 5.4, Redis, Celery Beat	Priority queues, async workers, daily cron scheduler, dead-letter retry
AI Pipeline	LangGraph 0.2.60, LangChain Core	10-node deterministic state machine per invoice
LLM Gateway	LiteLLM 1.55.8	Provider-agnostic: Groq, Gemini, OpenAI, Ollama, Together, mock
Dispatch & Integrations	SMTP, Resend, gspread 6.1, Langfuse	Email delivery, Google Sheets bidirectional sync, LLM tracing
Data Persistence	SQLModel 0.0.22, SQLAlchemy 2.0	6 tables in SQLite (dev) / PostgreSQL — invoices, audit, sent, queue, DLQ, activity

An invoice enters through ingestion (CSV upload or Sheets sync), is stored in the invoices table, and enqueued in Redis. A Celery worker picks it up and runs the LangGraph pipeline. The pipeline routes to auto-dispatch or human review depending on confidence and stage. After dispatch, audit entries are written to SQLite and optionally to Google Sheets. The activity_events table feeds the live dashboard feed every 5 seconds.

3. LangGraph Pipeline

Every overdue invoice runs through the same 10-node deterministic state machine compiled once per worker process. State flows through an InvoiceState TypedDict — every field has a named owner, nothing is implicit.

```
load_invoice
  → classify_stage
    → [30+ days ESCALATED] → human_queue
    → [Stage 1-4]
      → build_prompt
        → call_llm (LiteLLM + Langfuse trace)
          → validate_output
            valid? → confidence_check
            retry? → build_prompt (max 2x)
            failed? → fallback_template
              ↓
            confidence_check
            HIGH + Stage 1-3 → dispatch_email
            LOW or Stage 4 → human_queue
              ↓
            write_audit → END
```

Why LangGraph

- ▶ InvoiceState TypedDict is the explicit contract between all nodes — no hidden state, no implicit passing
- ▶ Conditional edges map escalation logic directly to graph structure — no nested if-else chains
- ▶ Retry loop is bounded by state (`retry_count ≤ 2`) — cannot run indefinitely
- ▶ Each node is a pure function — independently testable and clearly visible in Langfuse traces
- ▶ vs CrewAI/AutoGen: single-pass pipeline, not a multi-agent coordination problem

Node Responsibilities

Node	Responsibility
load_invoice	Fetches invoice from DB by ID. Serialises to InvoiceRecord dict for state.
classify_stage	Maps <code>days_overdue</code> → stage 1-4 or ESCALATED. 30+ days → <code>requires_human = True</code> immediately.
build_prompt	Pulls stage-specific template from prompt registry. Injects all invoice fields. Supports <code>tone_override</code> (1-4).
call_llm	Calls LiteLLM with JSON output mode enforced. Strips markdown fences. Wraps in Langfuse trace. Records model/tokens/latency.
validate_output	Pydantic parse + hallucination cross-check + placeholder detection. Increments <code>retry_count</code> on failure.
confidence_check	Scores output. Routes to human if: HIL enabled AND (low confidence OR stage 4) OR <code>auto_dispatch</code> off.
dispatch_email	Calls dispatcher, persists SentEmailTable row, bumps <code>followup_count</code> , pushes activity event.

human_queue	Deduplicates pending entries. Creates HumanQueueTable item. Flags 30+ day invoices as DISPUTED.
fallback_template	Deterministic mock LLM output. Sets fallback_used = True. Always routes to human review.
write_audit	Persists AuditTable entry. Optionally appends to Google Sheets audit tab and updates invoice status row.

4. Escalation Logic

Every invoice is classified by days overdue into one of five stages. Each stage maps to a distinct tone, key message, and CTA — enforced in the prompt and validated post-generation via Pydantic schema.

Stage	Days Overdue	Tone	Key Message / CTA	Routing
Stage 1	1–7	Warm & Friendly	Gentle reminder, assume oversight — pay now link	Auto-dispatch
Stage 2	8–14	Polite but Firm	Payment pending, request confirmation date	Auto-dispatch
Stage 3	15–21	Formal & Serious	Credit term impact, respond within 48 hours	Auto-dispatch (high-priority queue)
Stage 4	22–30	Stern & Urgent	Final notice, 24hr deadline, legal next	Human review when HIL enabled
Escalated	30+	No auto email	Requires manual finance/legal review	Forced human queue → status DISPUTED

Stage 4 always routes to human review regardless of confidence score when human-in-the-loop is enabled. 30+ day invoices skip email generation entirely — the pipeline sets invoice status to DISPUTED and sends directly to the review queue with no email drafted.

5. Email Generation & Validation

Prompt Structure

The LLM receives a structured system prompt with 10 strict rules, a stage-specific user prompt with all invoice fields injected, and a required JSON output schema. No free-form instruction — every constraint is explicit and code-enforced.

System Prompt (abbreviated)

```
You are an enterprise finance collections assistant.

STRICT RULES:
1. Never hallucinate invoice data. Use ONLY values provided.
2. invoice_id_used MUST equal exactly: {invoice_id}
3. client_name_used MUST equal exactly: {client_name}
4. amount_used MUST equal exactly: {amount}
5. days_overdue_used MUST equal exactly: {days_overdue}
6. Output ONLY valid JSON. No markdown. No preamble.
7. Tone MUST match: {tone_requirement}
8. Never mention legal action unless stage is STAGE_4.
... (10 rules total)
```

Anti-Hallucination Enforcement

The LLM must echo back `invoice_id_used`, `client_name_used`, `amount_used`, and `days_overdue_used` in its JSON output. These are cross-checked against the source record in code. Any mismatch fails validation and triggers a retry.

This was added after noticing that in batch runs the model would occasionally transpose invoice IDs across clients. Relying on prompt instructions alone was insufficient — code-level cross-validation eliminated the failure mode entirely.

Validation Pipeline

#	Check	Failure action
1	Pydantic schema parse	Field type errors, missing fields, invalid tone enum → validation failure
2	Hallucination cross-check	4-field comparison vs source invoice → failure on any mismatch
3	Placeholder detection	[INSERT], TBD, PLACEHOLDER in body → validation failure
4	Retry loop	Up to 2 re-generations on validation failure before falling back
5	Fallback template	Deterministic output when retries exhaust → always routed to human review
6	Confidence threshold	Below 0.75 (configurable) → human review queue regardless of validity

Every generated email contains: client name, invoice number, amount, due date, days overdue, and a dynamic payment link — all pulled from data, never invented.

6. Dashboard

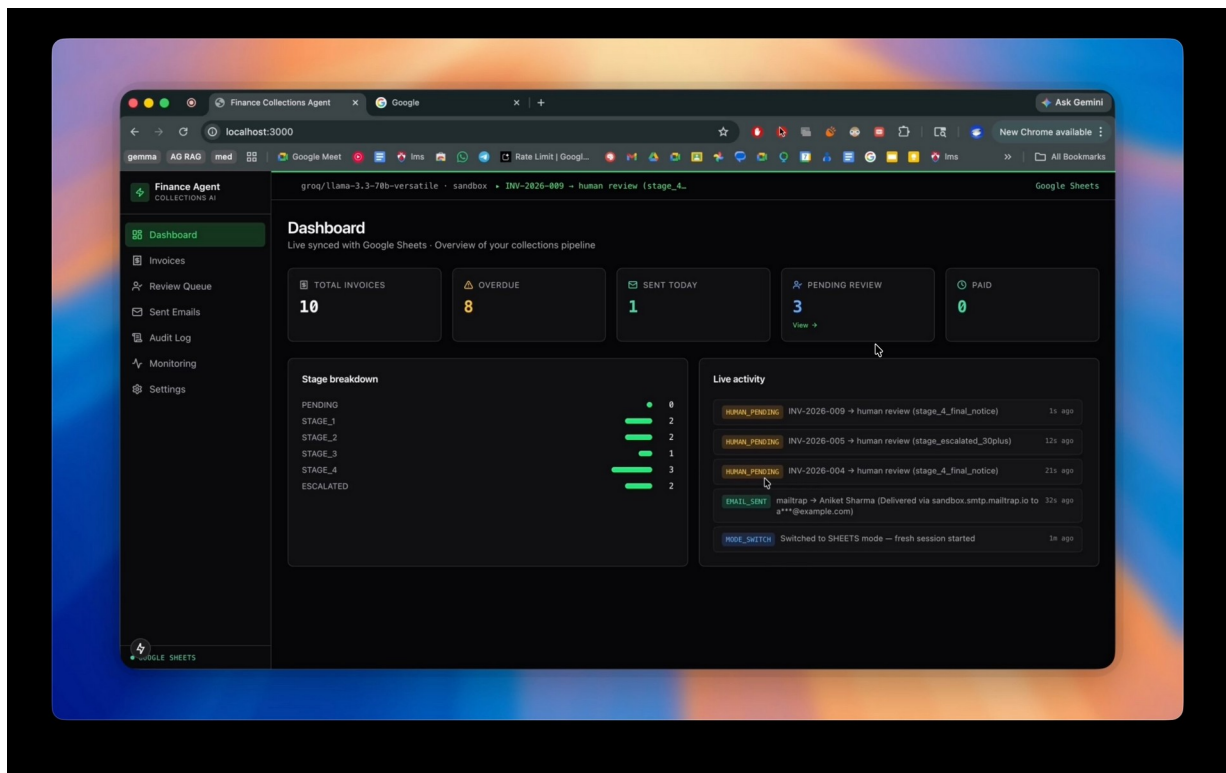


Figure 2 — Dashboard: KPI cards, stage breakdown, live activity feed (5-second polling)

The main dashboard shows total invoices, overdue count, emails sent today, pending human review, and paid count. The stage breakdown chart shows distribution across escalation stages. The live activity feed polls every 5 seconds and streams all pipeline events in real time — emails sent, human queue entries, mode switches, auth failures, dead letters.

Onboarding

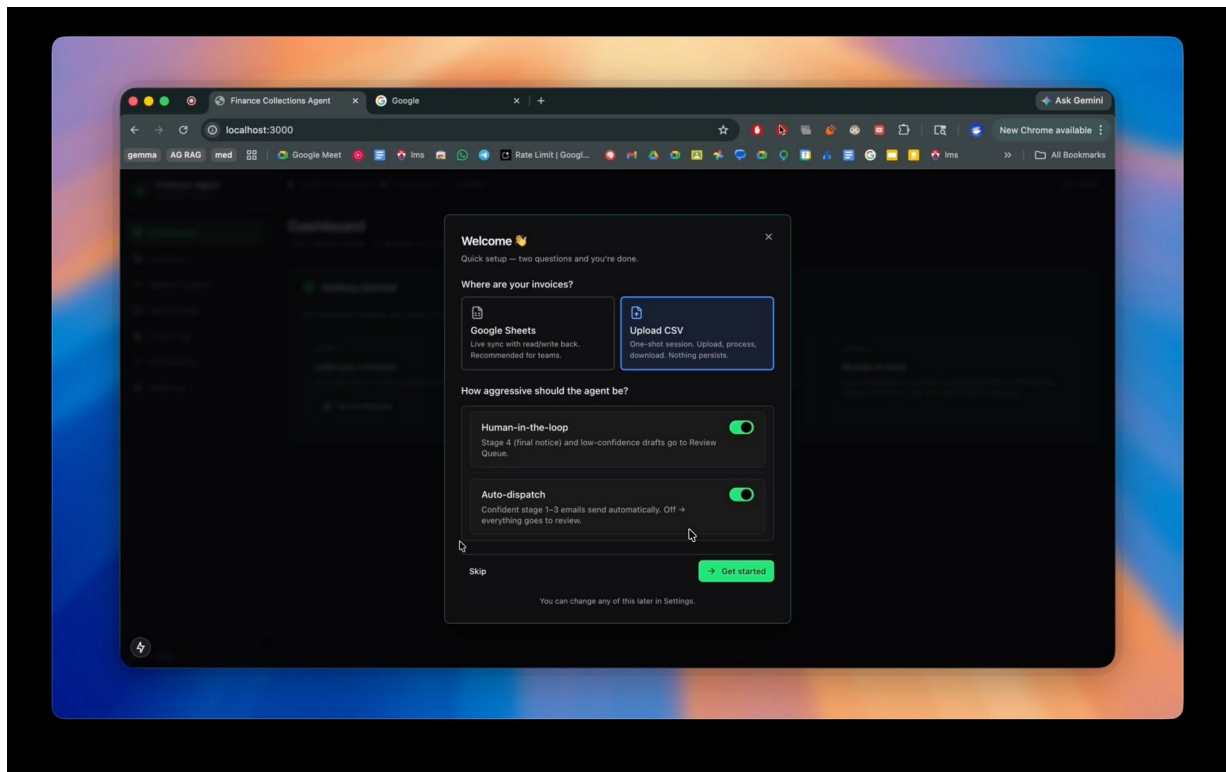


Figure 3 — Welcome modal: data source and behaviour toggles configured on first boot

The welcome modal collects the two key operational decisions on first boot: where invoices come from (CSV upload or Google Sheets) and how aggressive the pipeline should be (human-in-the-loop and auto-dispatch). Both are changeable from Settings at any time without restart.

7. Human Review Queue

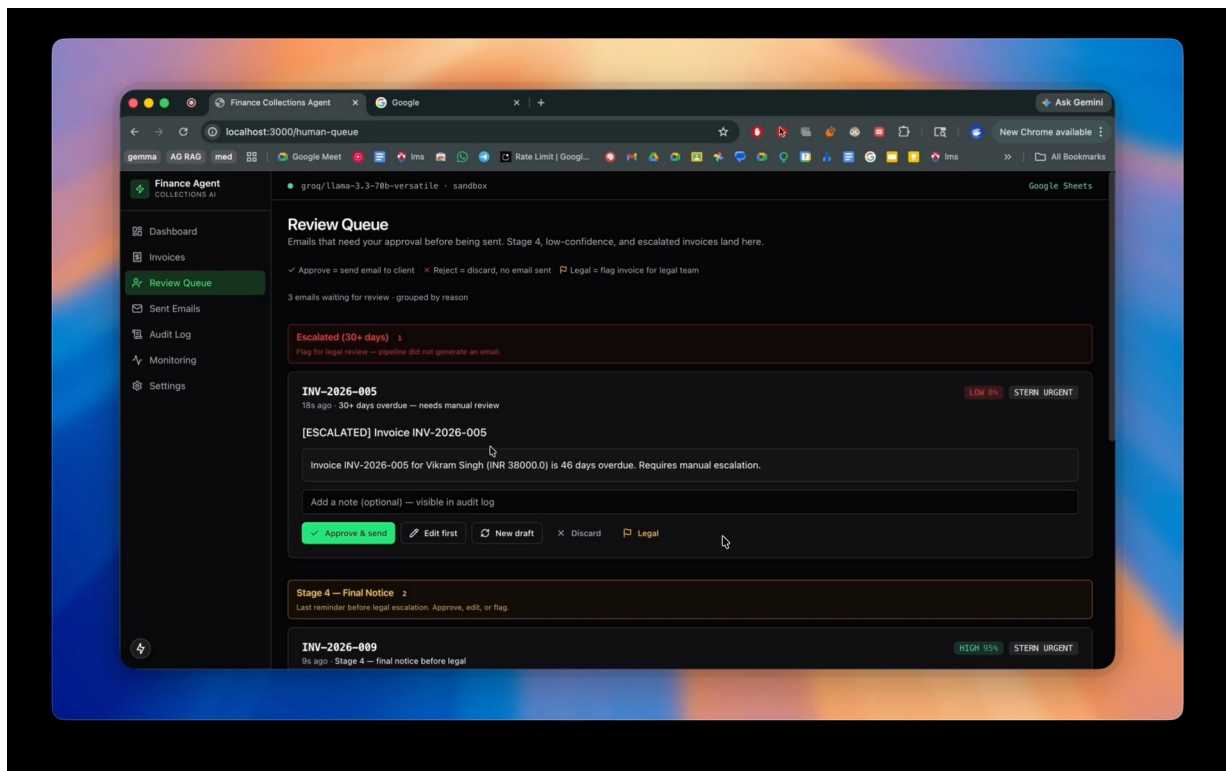


Figure 4 — Review Queue: escalated invoices (30+ days) and Stage 4 final notices grouped by reason

Low-confidence drafts, Stage 4 notices, and all 30+ day escalated invoices land here. Items are grouped by reason so reviewers can triage by urgency. Each item shows the confidence badge (HIGH/MEDIUM/LOW), tone label, full email body preview, and an optional reviewer note field that is persisted in the audit trail.

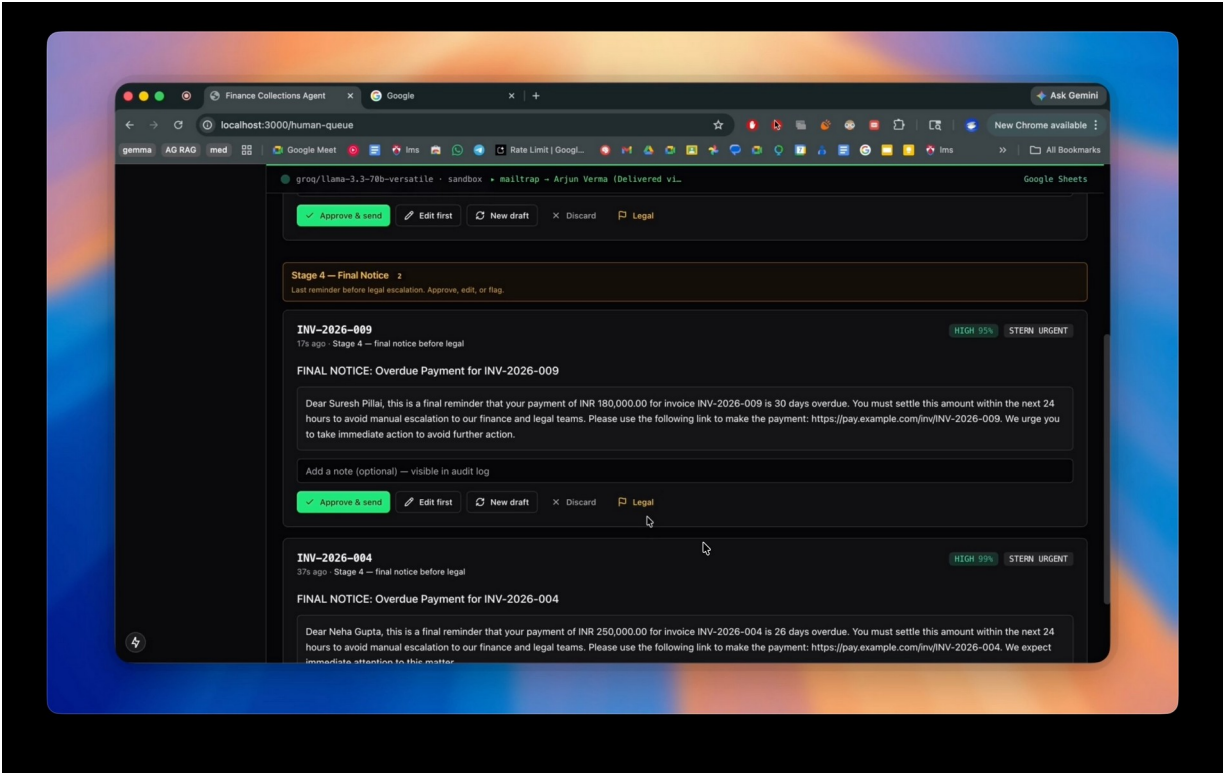


Figure 5 — Stage 4 drafts at HIGH 95% and HIGH 99% confidence — full body visible for review before dispatch

Reviewer Actions

Action	What happens	Audit outcome
Approve & send	Dispatches draft as-is via current email mode	HUMAN_APPROVED, delivery_status = SENT or SANDBOX
Edit first	Inline editor, edited version dispatched, original body preserved	HUMAN_APPROVED, edited_by_human = True
New draft	Re-runs pipeline for this invoice, previous draft superseded	Previous item resolved as regenerated
Discard	No email sent, invoice flagged, removed from queue	HUMAN_REJECTED, no SentEmail row created
Legal	Sets invoice status to LEGAL, removes from email automation	STAGE_ESCALATED, invoice.status = LEGAL

8. Email Delivery

Three modes switchable live from Settings without a restart. Default is mock — no emails are sent until explicitly configured.

Mode	Provider	Behaviour
Mock	None (log only)	Emails generated and persisted, no network call. Used for development.
Sandbox	Mailtrap SMTP	Real SMTP delivery to a private Mailtrap inbox. Real rendering, deliverability metadata, but the client never receives anything. Used for demos and staging.
Live	Resend API	Emails sent to real recipients. Requires a verified sender domain with SPF, DKIM, and DMARC. Provider message ID persisted for delivery tracking.

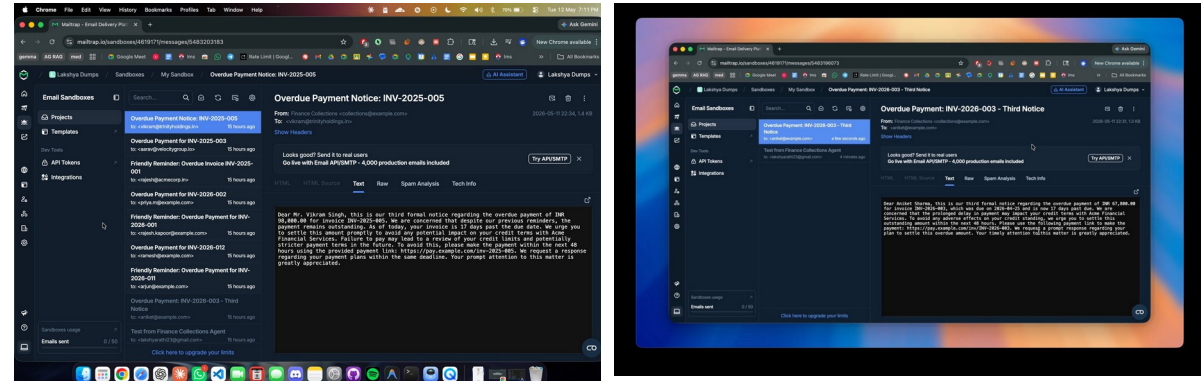


Fig 6 — Mailtrap inbox: multiple stages received

Fig 7 — Stage 3 formal notice rendered

Sent Emails Log

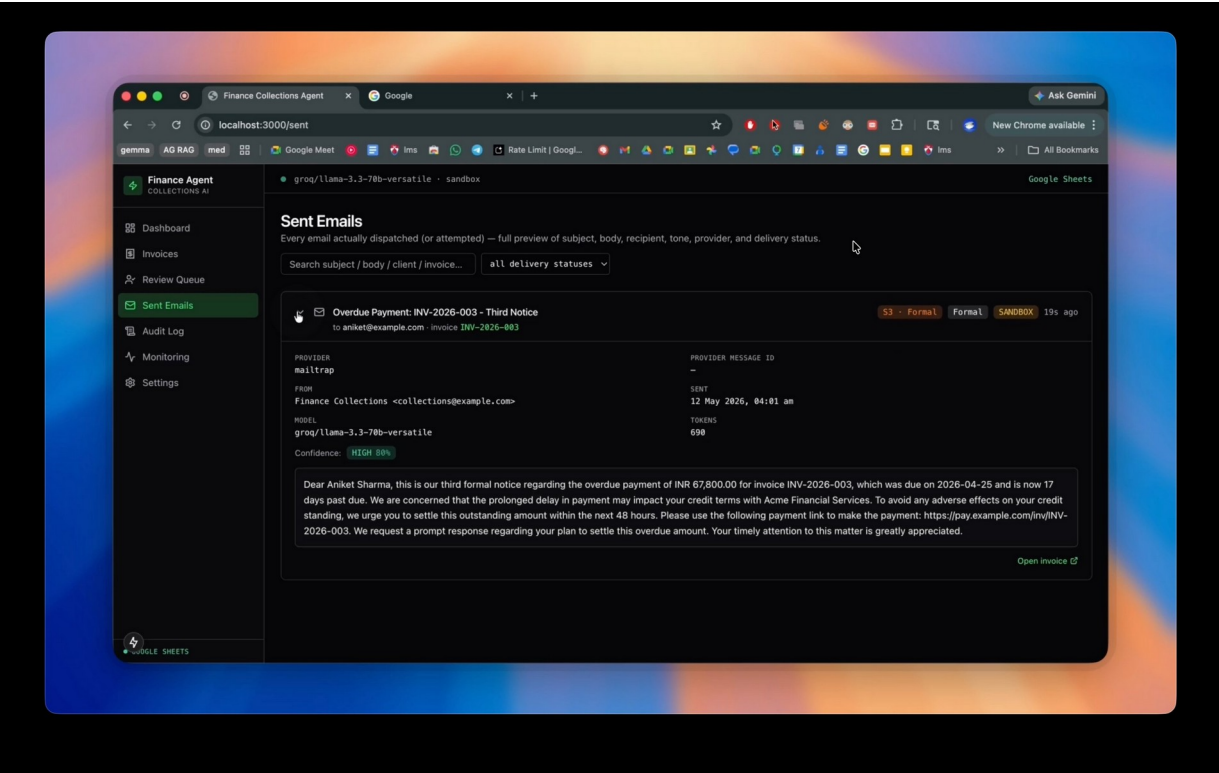


Figure 8 — Sent Emails page: provider, model, tokens, confidence badge, expandable full body

Every dispatch attempt — success or failure — is persisted in the sent_emails table with provider name, provider message ID, model used, token count, latency, confidence score, and human-approved flag. This table is the single source of truth for what was actually sent.

9. Google Sheets Integration

invoice_id	client_name	client_email	amount	currency	due_date	payment_link	notes	status	stage
INV-2024-001	Rajesh Kumar	rajesh.kumar@gmail.com	45000	INR	2024-05-08	https://example.com/invoice/INV-2024-001	Net 30 terms, 3 days overdue (Stage 1)	ACTIVE	STAGE
INV-2024-002	Priya Mehta	priya.mehta@gmail.com	12500	INR	2024-05-09	https://example.com/invoice/INV-2024-002	Marketing retainer Q1: 8 days overdue (Stage 2)	ACTIVE	STAGE
INV-2024-003	Amit Sharma	amit.sharma@gmail.com	87500	INR	2024-04-25	https://example.com/invoice/INV-2024-003	Follow-up reminder: 18 days overdue (Stage 3)	ACTIVE	STAGE
INV-2024-004	Neha Gupta	neha.gupta@gmail.com	25000	INR	2024-04-18	https://example.com/invoice/INV-2024-004	Large account, finance team Stage: 25 days overdue	ACTIVE	STAGE
INV-2024-005	Varun Singh	varun.singh@gmail.com	38000	INR	2024-03-27	https://example.com/invoice/INV-2024-005	Evaluation overdue, 40 days overdue (Stage Escalated)	ESCAL	STAGE
INV-2024-006	Suresh Patel	suresh.patel@gmail.com	180000	INR	2024-04-12	https://example.com/invoice/INV-2024-006	SLA breach, chase weekly, 20 days overdue (Stage URGENT)	URGENT	STAGE
INV-2024-017	Adarsh Verma	adarsh.verma@gmail.com	55000	INR	2024-05-10	https://example.com/invoice/INV-2024-017	1 day overdue (Stage 1)	ACTIVE	STAGE
INV-2024-012	Ramkishan Kumar	ramkishan.k@gmail.com	40000	INR	2024-04-01	https://example.com/invoice/INV-2024-012	10 days overdue (Stage 2)	ACTIVE	STAGE
INV-2024-013	Satish Dutt	satish.dutt@gmail.com	120000	INR	2024-04-20	https://example.com/invoice/INV-2024-013	21 days overdue (Stage 3)	ACTIVE	STAGE
INV-2024-015	Meeta Joshi	meeta.joshi@gmail.com	64000	INR	2024-03-15	https://example.com/invoice/INV-2024-015	Long overdue, 37 days overdue (Stage Escalated)	ESCALATED	STAGE

Fig 9 — Invoice source sheet (pulled every 30 min)

timestamp	email	subject	confidence_score	tokens_used	latency_ms	delivery_status	human_override	reason_validation_error_message	model_used
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini
2024-05-08T10:30:00Z	rajesh.kumar@gmail.com	Invoice # INV-2024-001	0.95	680	1500	True	False		gpt-4o-mini

Fig 10 — Audit log sheet (written after every run)

Finance teams often prefer to manage invoices in Google Sheets. The integration pulls invoice rows from a configured sheet on a 30-minute Celery Beat schedule, upserts them into the invoices table, and writes results back after processing.

Bidirectional Sync

- Pulls new/updated invoice rows into the invoices table every 30 minutes
- Writes one audit row per pipeline action: timestamp, stage, tone, confidence, tokens, latency, delivery status, model used
- Updates invoice status and stage in the source sheet after dispatch
- Auto-adds missing columns on first sync rather than failing
- Authenticated via GCP service account (credentials.json) — no OAuth user flow needed

A finance team can operate entirely from Google Sheets — the dashboard is purely for monitoring, approvals, and observability. Data ownership stays with the team.

10. LLM Choice & Rationale

Default: groq/llama-3.3-70b-versatile via LiteLLM

Criterion	Groq + Llama 3.3 70B
Cost	Free tier, no credit card — zero barrier for evaluators and contributors
Speed	~1–1.5s end-to-end via Groq LPU hardware. Fast enough for live regeneration from the dashboard.
JSON output mode	Native response_format: json_object support. Parsed directly into the GeneratedEmail Pydantic schema.
Output quality	Validated across 330+ pipeline runs in Langfuse. Strong adherence to the exact-field-echo requirement the hallucination check depends on.
Context window	128k tokens — system prompt + invoice context + output schema fits with headroom.
Portability	LiteLLM gateway means zero code changes to switch to any other provider.

Alternatives Considered

Alternative	Reason not chosen as default
llama-3.1-8b-instant (Groq)	Faster but less reliable JSON schema adherence on multi-field structured outputs in batch runs
gemini-1.5-flash	Comparable quality; slightly higher hallucination rate on exact-field-echo in testing
gpt-4o-mini (OpenAI)	Best overall quality but requires billing setup; unnecessary for this scope; also supported via env var
Ollama (local)	Fully offline option; supported and tested; useful for privacy-sensitive deployments

Switching Providers

Two environment variables. The model picker in Settings applies it live without a restart.

```
# Gemini
LLM_PROVIDER=gemini
LLM_MODEL=gemini/gemini-1.5-flash-latest
LLM_API_KEY=your_key

# OpenAI
LLM_PROVIDER=openai
LLM_MODEL=openai/gpt-4o-mini
LLM_API_KEY=your_key

# Ollama — fully local, no API key
LLM_PROVIDER=ollama
LLM_MODEL=ollama/llama3.1
LLM_BASE_URL=http://localhost:11434
```

11. Security

Six risk categories addressed. Each mitigation is implemented in code, not just documented.

Risk	Mitigation	Code location
Prompt injection	All client-supplied fields pass through a validator stripping patterns like "ignore previous instructions", "system:", "{{", "< " before reaching the prompt. JSON output mode prevents injection via the response path.	models/invoice.py (sanitize_client_name)
Hallucination	LLM must echo all invoice fields in its output. Cross-checked in code. Retry loop + deterministic fallback + confidence threshold + human routing for low-confidence outputs.	models/email_output.py (check_hallucination)
API key exposure	All secrets via .env and pydantic_settings.BaseSettings. .env.example committed with placeholders. .env in .gitignore. LLM API keys never reach the browser.	config.py, .env.example
Unauthorised access	X-API-Key header required on all /api/* routes. Auth failures logged to activity_events — visible in live feed for brute-force detection. SlowAPI rate limits. CORS locked to configured origins.	api/auth.py
PII in logs	Email addresses masked in all stdout log output (r***@domain.com). Full email body in DB only, never in stdout or external logs.	email/dispatcher.py (mask_email)
Email spoofing	Default is mock. Sandbox (Mailtrap) catches everything before real inboxes. Live mode via Resend requires verified sender domain with SPF, DKIM, and DMARC.	email/dispatcher.py

12. Observability

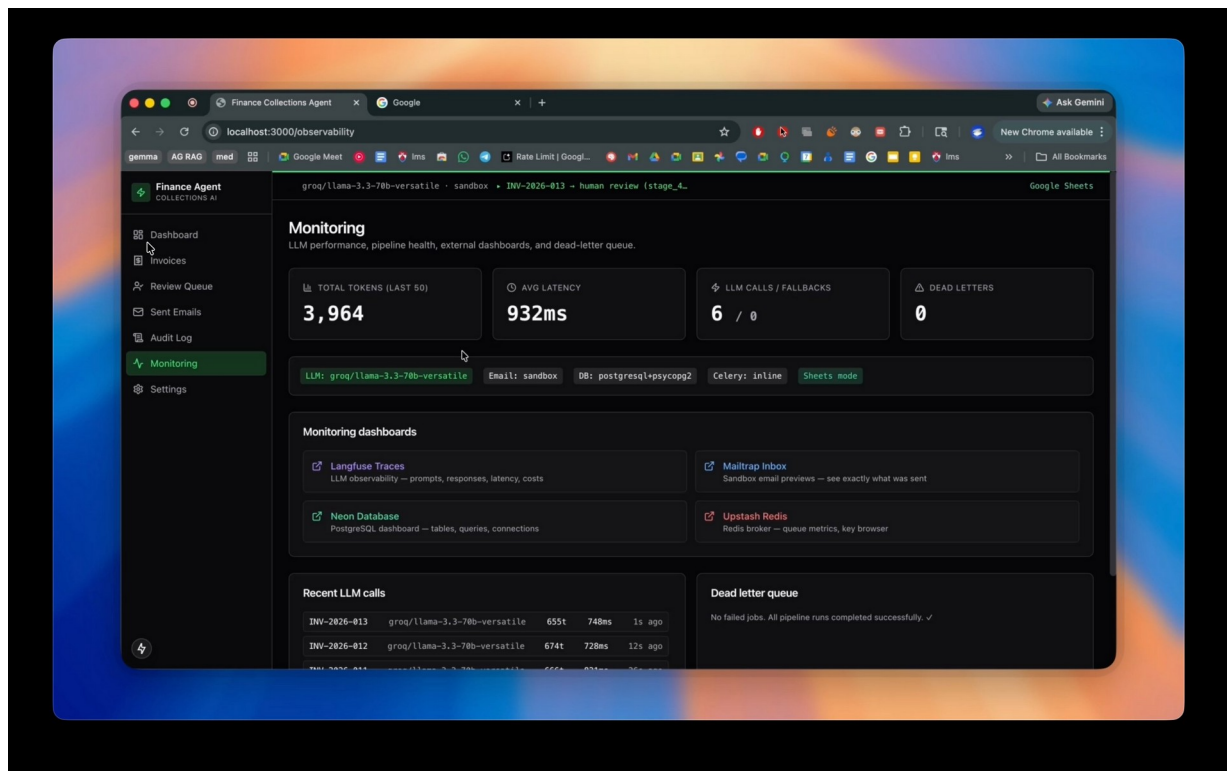


Figure 11 — Monitoring page: tokens, latency, LLM call/fallback ratio, dead-letter queue, external links

The Monitoring page shows total tokens across recent runs, average LLM latency, the ratio of successful LLM calls to fallback template uses, and the dead-letter queue with a one-click retry. External dashboard links — Langfuse, Mailtrap, Neon, Upstash — are wired directly into the page.

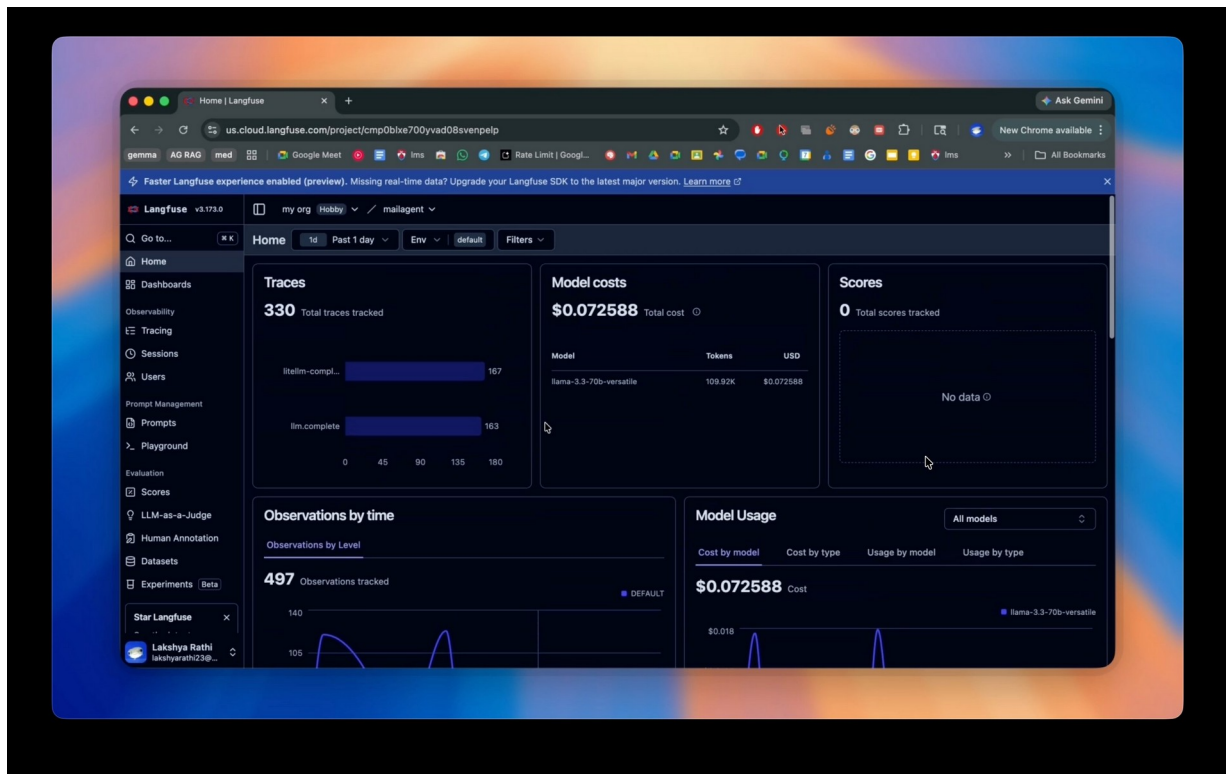


Figure 12 — Langfuse dashboard: 330 traces, \$0.073 total cost, 109k tokens, groq/llama-3.3-70b-versatile

Every LLM call is wrapped in a Langfuse trace tagged with `invoice_id` and `stage`. The trace captures the raw system prompt, raw user prompt, raw model response, token counts, and latency — allowing full audit of what the model received and produced for any invoice at any point in time.

Observability Stack

- ▶ Langfuse — per-call LLM traces with prompt versions, token costs, latency, and validation outcomes
- ▶ Celery Flower — queue depth, worker status, task history, retry counts (available at `:5555` via `make flower`)
- ▶ Per-invoice Timeline tab — unified chronological feed of audit entries, sent emails, and activity events
- ▶ Activity feed — real-time event stream from `activity_events` table, polled every 5 seconds
- ▶ FastAPI /docs — full OpenAPI schema for all endpoints

13. Settings & Runtime Controls

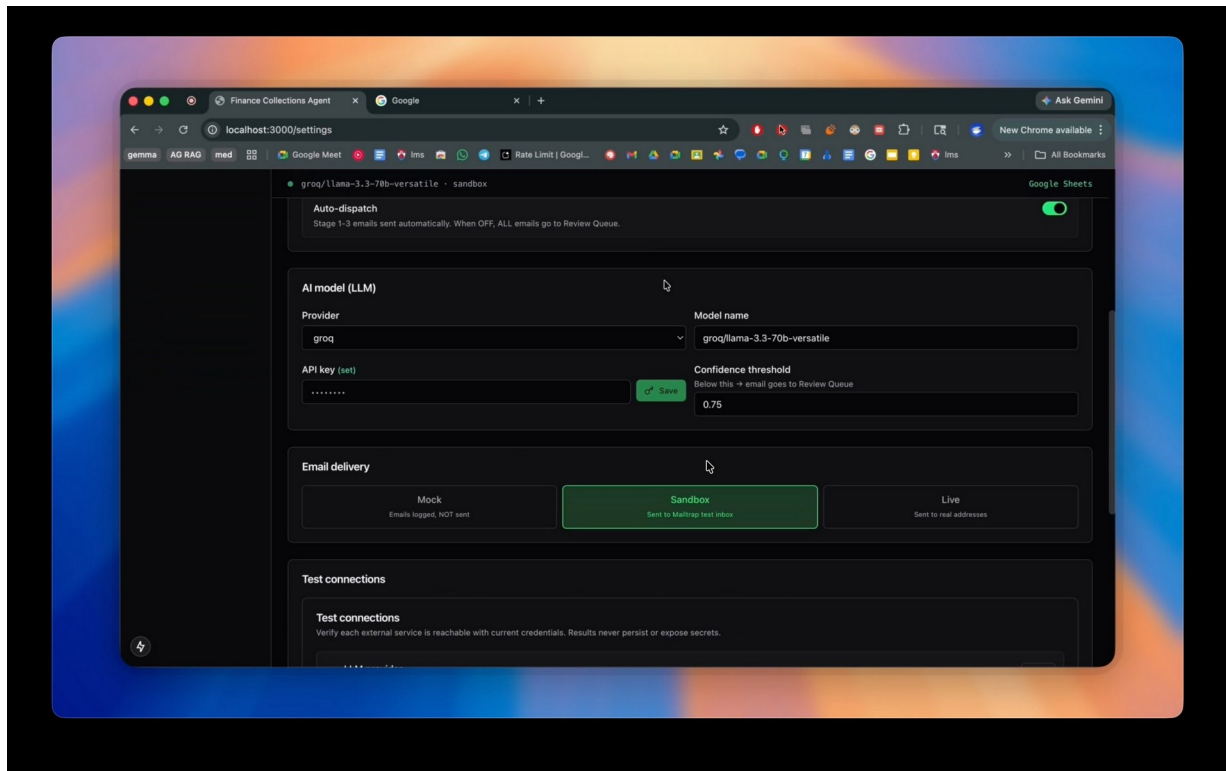


Figure 13 — Settings: LLM model picker, confidence threshold, email delivery mode, test connections

All operational settings are live-configurable from the Settings UI or via PATCH `/api/config` without a backend restart. The settings panel also includes a Test Connections pane that makes a real round-trip call to each external service independently — LLM, email, Redis, and Google Sheets — showing status and latency inline without persisting data.

Setting	Options	Effect
LLM provider	groq, gemini, openai, ollama, together, mock	Switches active LLM gateway immediately
LLM model	Any LiteLLM model string	Applied to next pipeline run
Confidence threshold	0.0 – 1.0 (default 0.75)	Below this: email goes to human review regardless of validity
Human-in-the-loop	true / false	On: stage 4 + low confidence → review queue. Off: all auto-dispatch.
Auto-dispatch	true / false	Off: every email queued for human approval regardless of confidence
Email mode	mock / sandbox / live	Switches delivery backend immediately
Data source	csv / sheets	Controls whether ingestion uses file upload or Sheets sync

14. Running the System

Zero Dependencies — Local Development

Default mode: mock LLM, logged emails (no sends), inline Celery tasks, SQLite. No external accounts needed.

```
git clone https://github.com/laksh-ya/finance-follow-up
cd finance-follow-up
cp .env.example .env
make install
make dev

# Frontend → http://localhost:3000
# API docs → http://localhost:8000/docs
```

Real LLM + Sandbox Email (~10 min setup)

```
# Add to .env
LLM_PROVIDER=groq
LLM_MODEL=groq/llama-3.3-70b-versatile
LLM_API_KEY=gsk_... # console.groq.com – free, no card

EMAIL_MODE=sandbox
MAILTRAP_HOST=sandbox.smtp.mailtrap.io
MAILTRAP_PORT=2525
MAILTRAP_USER=... # mailtrap.io – free inbox
MAILTRAP_PASS=...
```

Real Async Queue (Celery + Redis)

```
CELERY_EAGER=false
REDIS_URL=redis://default:token@host.upstash.io:6379 # upstash.com free tier

make worker # separate terminal – processes task queues
make beat # separate terminal – cron scheduler
make flower # optional – Celery UI at :5555
```

Full Docker Stack

```
make docker-up
# Starts: Redis, Backend, Celery Worker, Celery Beat, Flower, Frontend
```

Additional Services

```
# PostgreSQL (Neon or Supabase – both free)
DATABASE_URL=postgresql+psycopg2://user:pass@host/db?sslmode=require

# Langfuse tracing (50k events/month free)
LANGFUSE_ENABLED=true
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...

# Google Sheets
```

```
SHEETS_ENABLED=true  
SHEETS_CREDENTIALS_PATH=credentials.json  
SHEETS_INVOICE_ID=<sheet-id-from-url>  
SHEETS_AUDIT_ID=<audit-sheet-id>
```

All Make Commands

```
make help          # all targets  
make install       # backend + frontend deps  
make dev           # both servers  
make worker        # Celery worker  
make beat          # Celery Beat scheduler  
make flower        # Flower UI at :5555  
make seed          # seed demo invoices  
make reset         # wipe DB  
make test-llm      # test LLM connectivity  
make test-email    # send test email  
make typecheck     # TypeScript check  
make docker-up/down
```